

Laboratorio IA 4: Percepción Local

Reporte de experimento, código y conclusiones

Sebastián Machado Parra

20 de septiembre de 2026

1. Objetivo de la sesión

Transformar el robot de la clase 3 (agente open loop, con una ruta fija de 18 comandos) en un agente reactivo closed loop: la evolución ya no busca una ruta concreta, sino una política que observa las celdas vecinas y decide el siguiente movimiento durante la ejecución. El ADN deja de representar una secuencia de pasos y pasa a representar una tabla de reglas observación → acción.

2. Experimento guiado: tres semillas

Se ejecutó el modo comparativo (solo mutación vs. mutación + cruce) con las semillas 7, 21 y 42, usando el código original del laboratorio (ADN de 16 genes, PASOS_MAXIMOS = 18). Nota: por el diseño del script, -- modo comparar usa la semilla base para el método de mutación y semilla+1 para el método de cruce.

Semilla	Método	Llegó	Generaciones	Puntaje	Choques	Diversidad
7	Solo mutación	Sí	5	2556	0	66/100
7*	Mutación + cruce	Sí	4	2564	0	95/100
21	Solo mutación	Sí	8	2556	0	76/100
21*	Mutación + cruce	Sí	1	2556	0	100/100
42	Solo mutación	Sí	12	2556	0	76/100
42*	Mutación + cruce	Sí	7	2556	0	96/100

* El método de cruce corrió en realidad con semilla+1 (8, 22 y 43 respectivamente), no con la misma semilla que mutación.

En las 6 corridas el robot llegó a la meta sin ningún choque. Mutación + cruce convergió siempre en igual o menos generaciones que solo mutación, y con mayor diversidad final — el cruce combina material genético de más individuos, lo que acelera la búsqueda a costa de mantener más variedad en la población.

3. Preguntas para discutir

1. ¿Qué diferencia hay entre evolucionar una ruta y evolucionar una política?

En la clase 3 el ADN codificaba una secuencia fija de comandos (una ruta completa) que se ejecutaba tal cual, sin importar lo que pasara en el camino — era un agente open loop. En este laboratorio el ADN codifica una tabla de reglas (observación → acción) que se consulta en cada paso; la misma regla puede usarse muchas veces y en cualquier posición donde se repita esa observación. Evolucionar una ruta optimiza para un recorrido específico; evolucionar una política optimiza para responder a situaciones, sin importar en qué momento del recorrido aparezcan.

2. ¿Por qué una política local puede entrar en un ciclo?

Porque la observación (vecinos bloqueados, y en nuestra versión también dirección a la meta y memoria) no identifica de forma única la posición del robot: dos celdas distintas pueden producir exactamente el mismo patrón de observación. Si la regla asociada a esa observación lleva al robot de vuelta a una celda donde ya estuvo — y esa celda produce la misma observación otra vez —, el robot puede quedar atrapado alternando entre las mismas posiciones indefinidamente, sin memoria suficiente para 'saber' que ya pasó por ahí. En nuestras pruebas de la modificación 3 no llegamos a observar ciclos reales, pero sí el caso límite: cuando el presupuesto de pasos es insuficiente, el resultado es agotamiento del tiempo en vez de un ciclo evidente.

3. ¿Qué información pierde el robot al no conocer su posición exacta?

Pierde la noción global de dónde está dentro de la cuadrícula, cuánto ha avanzado, y si ya visitó una celda antes. Solo sabe qué hay inmediatamente a su alrededor (y, con las modificaciones 1 y 2, hacia qué lado general queda la meta y qué hizo en el paso anterior), pero no un mapa completo ni su historial de posiciones. Esto le impide

distinguir, por ejemplo, dos zonas del tablero que 'se ven' igual desde su sensor local aunque estén en lugares completamente distintos.

4. ¿El cruce combina rutas completas o reglas de percepción? ¿Qué efecto puede tener?

En la clase 3 el cruce combinaba fragmentos de una ruta completa (los primeros k pasos de un padre con el resto del otro). Aquí el cruce combina reglas de percepción: toma las primeras k reglas de un padre y el resto de reglas del otro. El efecto es distinto en cada caso: en una ruta, cortar a la mitad puede producir una secuencia de movimientos incoherente justo en el punto de corte; en una tabla de reglas, cada regla es independiente de las demás (no hay continuidad temporal que romper), así que el cruce puede combinar reglas buenas de ambos padres sin que eso implique necesariamente una política incoherente. Esto es consistente con lo observado: mutación + cruce convergió igual o más rápido que solo mutación en la mayoría de nuestras corridas.

5. ¿Una política que funciona en este mapa funcionaría necesariamente en otro mapa?

No necesariamente, y lo confirmamos de forma experimental en la modificación 4: la misma política que llegaba perfectamente a la meta en el mapa original falló en las 3 semillas probadas al cambiar la distribución de obstáculos. Esto ocurre porque la política asocia acciones a patrones de observación que dependen de dónde están los obstáculos; al mover los obstáculos, esos mismos patrones aparecen en otros lugares del tablero, donde la acción aprendida ya no es la correcta.

4. Reto de modificación

Se realizaron las cuatro modificaciones obligatorias sobre el código original, de forma acumulativa (cada modificación se construyó sobre la anterior).

4.1 Modificación 1 — Dirección relativa de la meta

Se agregó una función `direccion_relativa(posicion)` que indica si la meta está abajo/derecha (1) o arriba/izquierda (0) respecto a la posición actual, en dos bits adicionales. La observación pasó de 4 bits (16 posibilidades) a 6 bits (64 posibilidades), y `LONGITUD_ADN` se amplió de 16 a 64 genes.

Semilla	Método	Generaciones (antes)	Generaciones (con mod. 1)	Diversidad (con mod. 1)
7	Solo mutación	5	10	98/100
7	Mutación + cruce	4	11	100/100
21	Solo mutación	8	5	99/100
21	Mutación + cruce	1	9	100/100
42	Solo mutación	12	12	100/100
42	Mutación + cruce	7	9	100/100

Efecto observado:

En 5 de 6 casos el número de generaciones aumentó o se mantuvo igual; solo mejoró en un caso (solo mutación, semilla 21). Cuadruplicar el ADN (16→64 genes) amplía el espacio de búsqueda: hay más reglas que ajustar y muchas casi nunca se usan en este mapa específico, por lo que la convergencia no necesariamente se acelera aunque la información disponible sea mayor.

4.2 Modificación 2 — Memoria de la última acción

Se incorporó memoria: `recorrer()` guarda la última acción ejecutada y la pasa a `decidir()` en el siguiente paso. El índice de observación combina el patrón local (64 valores) con la memoria (5 estados: U, D, L, R o 'ninguna' en el primer paso), llevando `LONGITUD_ADN` de 64 a 320 genes (64×5).

Semilla	Método	Generaciones	Puntaje	Choques	Diversidad
7	Solo mutación	9	2561	0	100/100
7	Mutación + cruce	3	2457	3	100/100
21	Solo mutación	4	2523	1	100/100
21	Mutación + cruce	9	2424	4	100/100

Semilla	Método	Generaciones	Puntaje	Choques	Diversidad
42	Solo mutación	19	2528	1	100/100
42	Mutación + cruce	3	2424	4	100/100

Efecto observado:

El hallazgo más claro es que mutación + cruce empezó a mostrar choques de forma consistente (3, 4 y 4), algo que nunca ocurría antes de esta modificación (siempre era 0). Con 320 genes el espacio de búsqueda es tan grande que el cruce encuentra soluciones que llegan rápido pero 'tropiezan' en el camino, en vez de políticas perfectamente limpias. Solo mutación también presentó algún choque en 2 de 3 semillas, aunque en menor medida.

4.3 Modificación 3 — Cambio del número máximo de pasos

Se modificó la constante PASOS_MAXIMOS (valor original: 18) probando un límite bajo (8), uno intermedio (12) y uno alto (40), manteniendo activas las modificaciones 1 y 2.

PASOS_MAXIMOS	Semilla	Llegó	Choques (mutación)	Choques (cruce)
8	7	No	0	0
12	7 / 21 / 42	No (las 3)	0	0
18 (original)	7 / 21 / 42	Sí (todas, con mod. 1+2)	0 – 1	3 – 4
40	7	Sí	2	5
40	21	Sí	2	3
40	42	Sí	6	10

Efecto observado:

PASOS_MAXIMOS actúa como una restricción dura del problema, no solo como un parámetro de ajuste fino. La distancia mínima (Manhattan) entre INICIO (0,0) y META (7,7) es de 14 pasos. Con 8 o 12 pasos, ninguna política pudo llegar jamás —ni siquiera tras 3000 generaciones—, porque el problema es geoméricamente irresoluble con ese presupuesto; curiosamente nunca se observaron ciclos reales (repetición de celdas), sino simple agotamiento limpio del tiempo disponible. Con un límite alto (40), en cambio, el efecto es contrario al esperado: al haber margen de sobra, la presión selectiva contra los choques se relaja (hasta 10 choques en un caso) porque una política puede darse el lujo de tropezar varias veces y aun así llegar a tiempo; además, es más probable que la población inicial aleatoria ya traiga una solución válida por pura casualidad (llegadas en la generación 0).

4.4 Modificación 4 — Segunda distribución de obstáculos

Se agregó un segundo conjunto OBSTACULOS_MAPA_2 y una función usar_mapa() controlada por el argumento --mapa (original/alternativo). Se entrenó una política en el mapa original y, sin volver a evolucionar, se cargó esa misma política (--cargar ... --mapa alternativo) para correrla sobre el mapa nuevo.

Semilla	Mapa original (entrenamiento)	Mapa alternativo (misma política)
7	Llegó — Puntaje 2561	No llegó — Puntaje 417 (18/18 pasos)
21	Llegó — Puntaje 2523	No llegó — Puntaje 417 (18/18 pasos)
42	Llegó — Puntaje 2528	No llegó — Puntaje 352 (13/18 pasos útiles)

Efecto observado:

En los 3 de 3 casos la misma política que resolvía perfectamente el mapa original fracasó por completo en el mapa alternativo. Esto confirma que la política no aprendió una estrategia general de navegación, sino una tabla de respuestas ajustada a la posición exacta de los obstáculos originales: al cambiar el mapa, una misma vista local (por ejemplo 'ningún vecino bloqueado') aparece en lugares distintos del tablero, y la regla asociada deja de tener sentido ahí. Es la limitación más clara de una política reactiva basada solo en percepción local — no generaliza a entornos nuevos.

5. Código

El archivo `robot_percepcion.py` entregado junto a este reporte incluye las modificaciones 1, 2 y 4 integradas directamente en el código (dirección relativa de la meta, memoria de la última acción, y el argumento `--mapa` para alternar entre las dos distribuciones de obstáculos). La modificación 3 se documenta variando la constante `PASOS_MAXIMOS`, tal como se muestra en la sección 4.3; el archivo entregado la deja en su valor original (18).

- `LONGITUD_ADN = 320` (64 observaciones locales $\times$ 5 estados de memoria)
- Nuevas funciones: `direccion_relativa()`, `indice_memoria()`, `observacion_local()`, `usar_mapa()`
- `recorrer()` ahora rastrea y retorna la memoria de la última acción en cada paso
- Nuevo argumento de línea de comandos: `--mapa {original, alternativo}`

6. Conclusión

Percibir le dio al robot la capacidad de adaptar su comportamiento en tiempo real: en vez de ejecutar un plan fijo que fallaba en cuanto aparecía un obstáculo inesperado, la política reactiva consulta su entorno inmediato en cada paso y decide en consecuencia, lo que le permitió resolver el mapa de forma consistente y, con las modificaciones 1 y 2, incorporar más contexto (dirección a la meta, memoria de la última acción) para tomar mejores decisiones. Sin embargo, esta percepción es estrictamente local: el robot nunca conoce su posición global ni el mapa completo, solo una vista de cuatro celdas vecinas más algunas señales derivadas. Esa limitación se hizo evidente en tres frentes distintos. Primero, ampliar la observación (modificaciones 1 y 2) no garantizó mejores políticas: en varios casos aumentó las generaciones necesarias o introdujo choques que antes no existían, porque un ADN más grande implica un espacio de búsqueda más difícil de explorar. Segundo, el límite de pasos (modificación 3) reveló que ninguna cantidad de inteligencia evolutiva puede superar una restricción física del problema —si el presupuesto de pasos es menor a la distancia mínima, la tarea es sencillamente irresoluble—, y que un presupuesto generoso relaja la presión contra los errores en vez de mejorar la solución. Tercero, y de forma más contundente, la prueba en un segundo mapa (modificación 4) demostró que la política no generaliza: memorizó respuestas para un entorno específico y falló por completo al cambiar la posición de los obstáculos. En conjunto, el laboratorio muestra que la percepción local es suficiente para producir comportamiento adaptativo dentro de un entorno fijo, pero no para producir una estrategia de navegación transferible a otros mapas.